



Evolution Strategies at the Hyperscale

[Bidipta Sarkar](#), [Mattie Fellows](#), [Juan Agustin Duque](#), [Alistair Letcher](#), [Antonio León Villares](#), [Anya Sims](#), [Dylan Cope](#), [Jarek Liesen](#), [Lukas Seier](#), [Theo Wolf](#), [Uljad Berdica](#), [Alexander David Goldie](#), [Aaron Courville](#), [Karin Sevegnani](#), [Shimon Whiteson](#), [Jakob Nicolaus Foerster](#)

2025.11.20

FLAIR(Oxford), WhiRL(Oxford), MILA(Quebec AI Inst), NVIDIA AI Tech Center, CIFAR AI Chair

딥논읽 7기 자연어처리팀
김보성(발표자), 정현석, 박민건, 황예진, 박주혜,
이예서, 임다예슬, 김유진, 이웅기, 김기환, 조해창



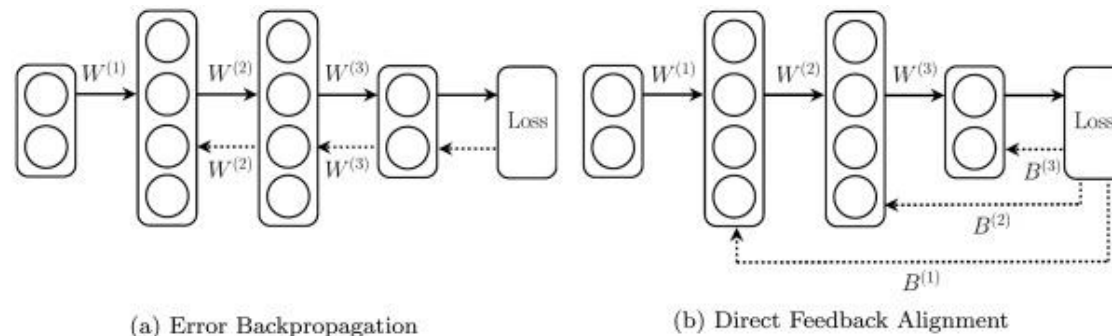
비판받는 GD(Gradient Descent) / BP (Backward Propagation)

그리고 대체 알고리즘들...

Feedback Alignment (Lillicrap et al., 2016)

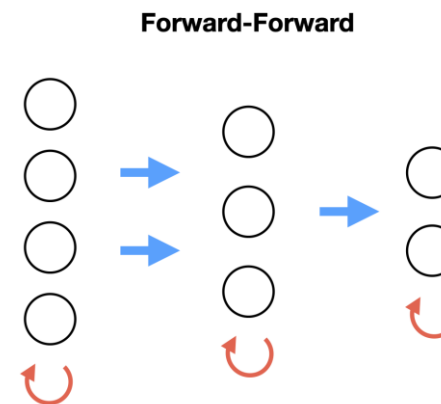
고정된 랜덤 가중치를 이용해 오차를 전파하는 학습법
생물학적 타당성에 집중.

인간의 뉴런은 앞뒤로 통하지 않는다.



Forward-Forward Algorithm (Hinton, 2022)

역전파 없이 긍정/부정 데이터의 두 번의 순전파로 학습
Global Error BP 없도록, 국소적(Local) 학습 방법



그러나 소규모 모델에서만 제한적 작동, 스케일링 불가.

Evolutionary Strategies(ES)

진화 전략

1.초기화 (Initialization)

- 기준 모델 파라미터 설정 (P_0)

2.모집단 생성 (Population Generation)

- 기준 모델에 노이즈(Perturbation)를 추가하여 N개의 변형 생성

3.평가 (Evaluation)

- 생성된 N개 모델의 성능(Fitness) 측정

4.선택 (Selection)

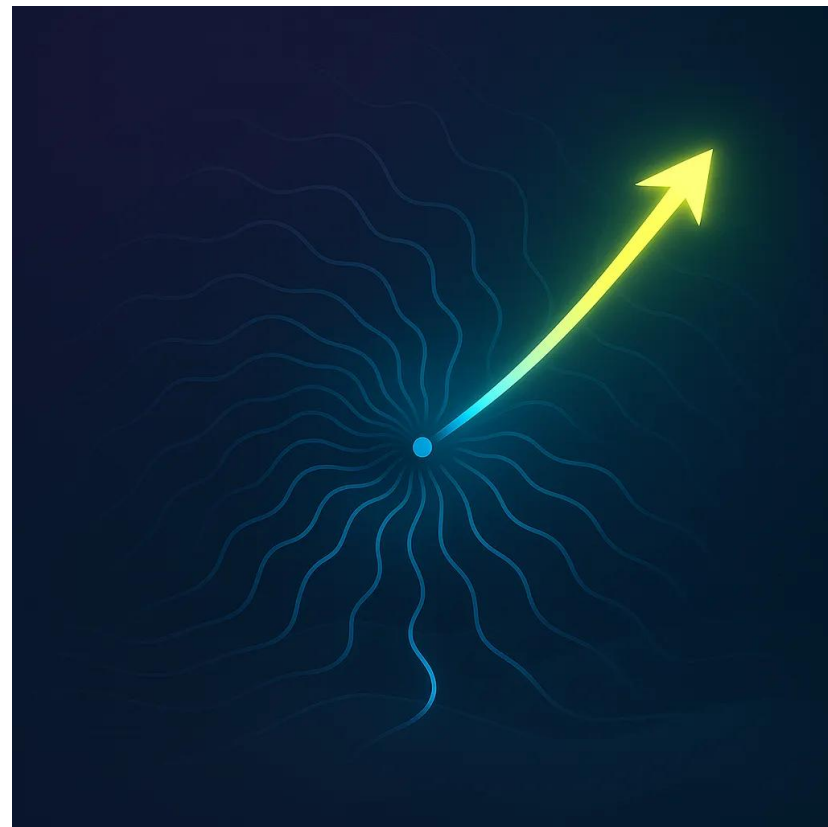
- 평가 점수 기준 상위 M개 모델 선별

5.업데이트 (Update)

- 선별된 모델들의 방향 벡터를 평균내어 파라미터 갱신 ($P_0 \rightarrow P_1$)

6.반복 (Iteration)

- 수렴 시까지 위 과정 반복



Evolutionary Strategies(ES)

진화 전략

주변 모든 곳으로 “꼬마 탐험가 ” 를 보낸다.

“여기 좋아요! ” 하는 꼬마 탐험가들이 많은 방향 이동

“별로예요...” 하는 꼬마 탐험가들은 외면한다.

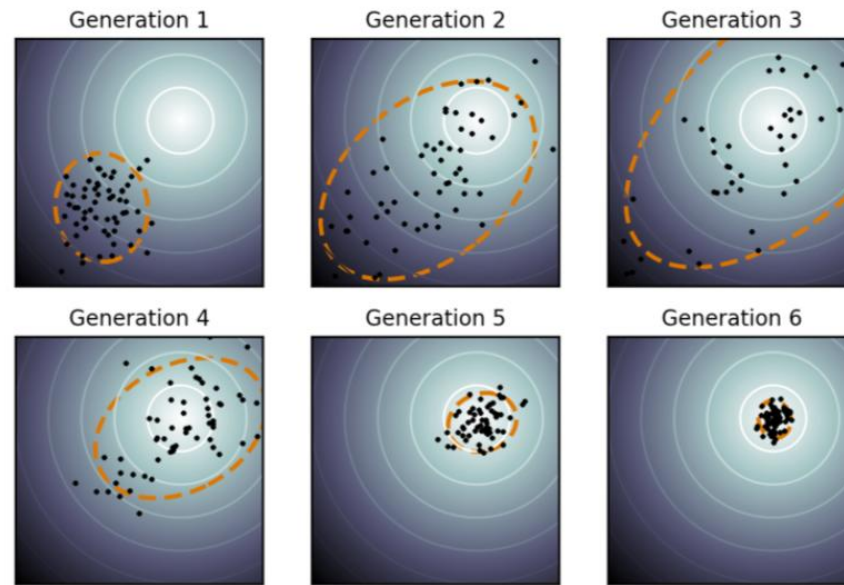
이동한 지점에서, 다시 꼬마 탐험가를 보낸다.

반복하면, 좋은 곳에 간다.



ES의 이점과 한계점

- Backpropagation을 하지 않는다.
 - Gradient 계산이 필요 없다.
 - Activation을 저장하지 않는다.
- 따라서 굳이 모델이 미분가능하지 않아도 된다.
 - TopK와 같은 Select 연산을 더 자연스럽게 사용 가능.
 - 연산 제한 X, Backward Kernel 불필요.
 - Precision Limit에서도 자유롭다. 즉 실행과 학습이 빠르다.
- 그럼 이거 왜 안 써요?
 - ➔ Noise = 모델 파라미터와 동일한 크기
 - ➔ Population 크기에 비례한 성능, 결국 꼬마의 수가 많아져야.
 - ➔ “스케일링 불가”



Wait a second...

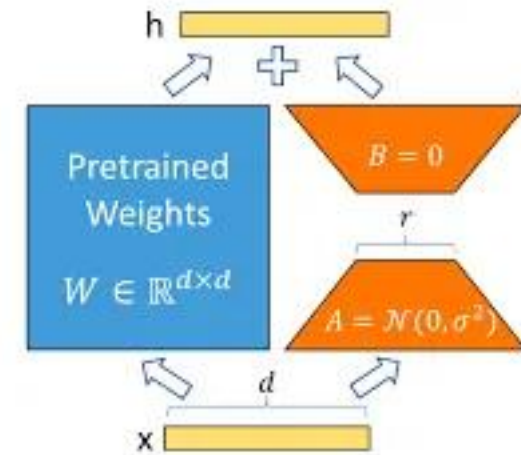
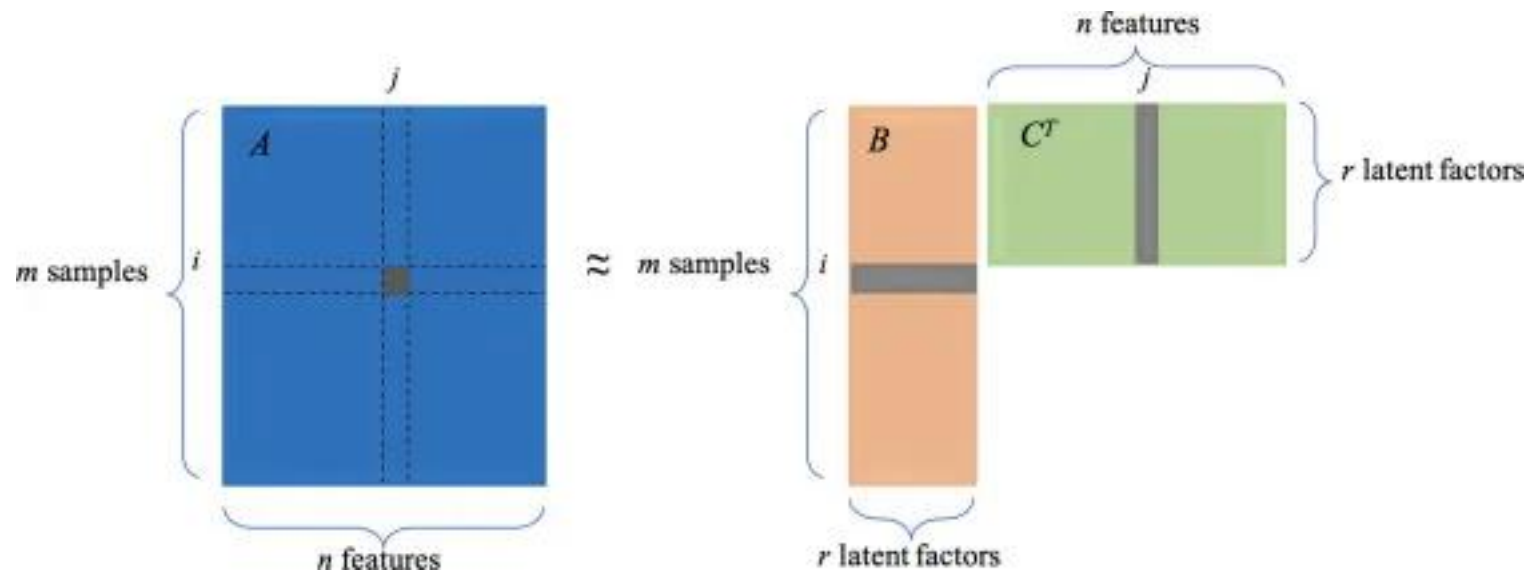


Figure 1: Our reparametrization. We only train A and B .

우리는 큰 행렬을 쪼개서 표현하고,
원본 가중치가 있을 때 무언가를 더해 변동을 주는 기법을 이미 알고 있다.
Tensor(Matrix) Decomposition / Low-rank Adaptation.

EGGROLL : Evolution Guided General Optimization via Low-rank Learning

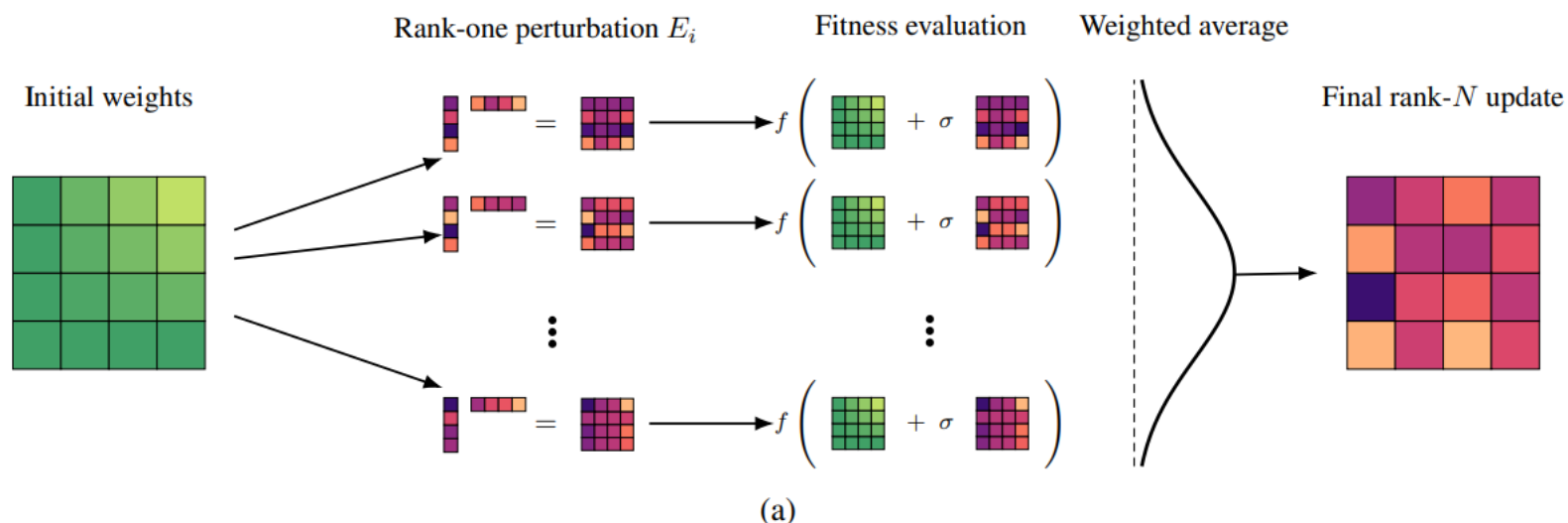


Figure 1: Schematic visualization of EGGROLL using N workers.

노이즈가 크다? Low-Rank Decomposition 한다.

작아졌다! Population을 늘릴 수 있다! 스케일링 가능해졌다!

15억 파라미터 모델도 GPU 1대로 학습이 잘 된다! 생각보다 빠르다!

Full-Integer Training도 잘 된다! RL도 잘 된다! Reasoning도 잘 된다! 신난다!

EGGROLL : Evolution Guided General Optimization via Low-rank Learning

1. 특정 랭크로 쪼갠 노이즈를 N 개 생성한다.
2. 해당 노이즈 (Perturbation/섭동)을 더한 모델들의 Fitness를 판단한다.
3. 해당 Fitness를 가중합의 가중치로, 각 섭동의 방향의 가중 평균을 구한다.
4. 해당 방향으로 Rate α 만큼 업데이트한다.
5. 반복한다.

Algorithm 1 EGGROLL($r, \alpha, \sigma, T_{\max}, N_{\text{workers}}$)

```
initialise  $\mu$  and workers with known random seeds  $\varsigma$ 
for  $T_{\max}$  timesteps do
  for Each worker  $i \in \{1, \dots, N_{\text{workers}}\}$  in parallel do
     $A_i \sim p(A_i), B_i \sim p(B_i)$ 
     $E_i \leftarrow \frac{1}{\sqrt{r}} A_i B_i^\top$ 
     $f_i \leftarrow f(\mu + \sigma E_i)$ 
  end for
  Workers share scalar fitness  $f_i$  with other workers
  for Each worker  $i \in \{1, \dots, N_{\text{workers}}\}$  in parallel do
    Reconstruct  $E_j$  for  $j \in \{1, \dots, N_{\text{workers}}\}$  from  $\varsigma$ 
     $\mu \leftarrow \mu + \alpha \frac{1}{N_{\text{Workers}}} \sum_{j=1}^{N_{\text{Workers}}} E_j f_j$ 
  end for
end for
```

EGGROLL : Evolution Guided General Optimization via Low-rank Learning

EGGROLL UPDATE: For each worker (in parallel), sample $A_{i,t} \sim p(A_{i,t})$, $B_{i,t} \sim p(B_{i,t})$ and form low-rank perturbation $E_{i,t} = \frac{1}{\sqrt{r}} A_{i,t} B_{i,t}^\top$. Update matrix parameters using:

$$\mu_{t+1} = \mu_t + \frac{\alpha_t}{N_{\text{workers}}} \sum_{i=1}^{N_{\text{workers}}} E_{i,t} f(M = \mu_t + \sigma E_{i,t}). \quad (8)$$

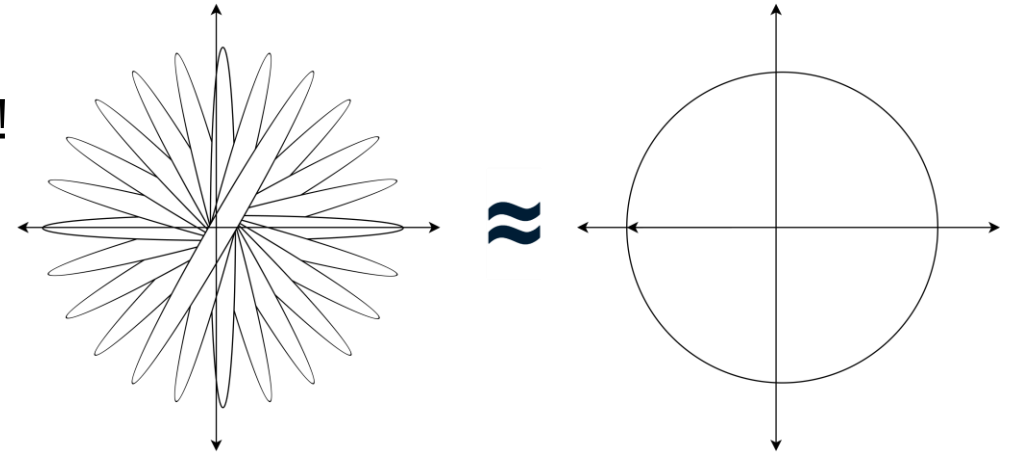
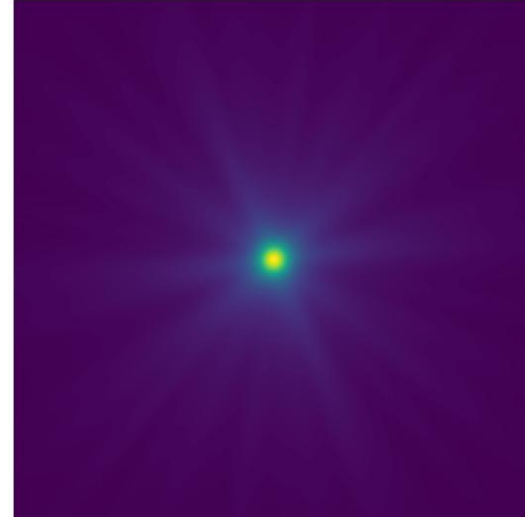
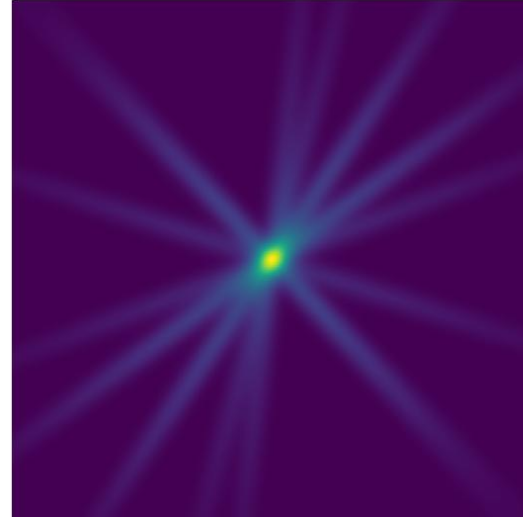
1. 생각보다 간단하다. 성능이 좋았던 방향으로 가중치를 부여한다는 것이 전부이다.
2. $r = 1$ 에서도 잘 된다. 이 경우, 배치 연산까지 가능해져 수 배는 빨라진다.
 1. N개 Population에 대해 $(N, 1, K) \rightarrow (N, K)$, (A, K) Input에 대해 $(A, K) * (K, N) \rightarrow (A, 1) * N$ (N개의 스칼라)
 2. 단순 LoRA Inference와 거의 유사한 속도.
3. 그런데, Rank가 1이면 단순 벡터가 된다. 벡터 * 벡터 = 스칼라로 Collapse된다.
4. 표현력이 급감하기 마련.
5. 어떻게 했지?

한번쯤 들어본 그 것

중심극한정리 : Central Limit Theorem

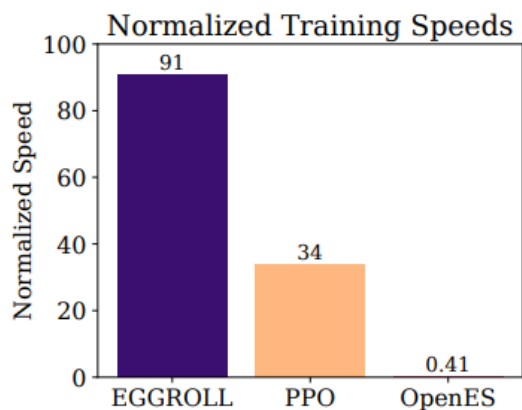
1. Full-Rank Perturbation E 가 원본 분포라고 하자.
2. $r=1$ A 와 B 를 가우시안에서 Sampling.
3. E 또한 가우시안에서 Sampling했다면,
 $r=1$ 짜리 섭동이 무한히 많아질 때 E 와 동일한 분포로 수렴한다.
(동일한 특성의 분포가 된다)
4. 즉, 원본 사이즈 Perturbation을 쓰는 것과 다를 것이 없다!

모집단의 분포 형태와 관계없이,
충분히 큰 표본들을 반복적으로 추출하여 얻은 표본 평균들의 분포는
표본의 크기가 커질수록 정규분포에 가까워진다.

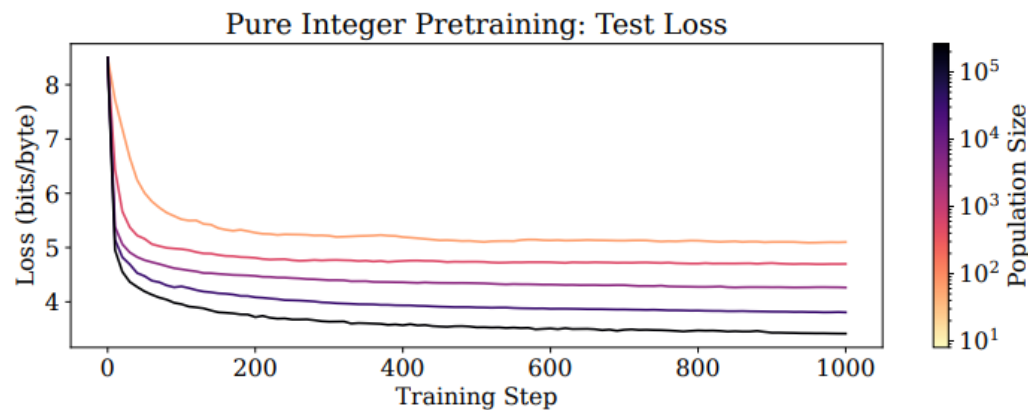


Experiments

Pure Integer Training



(a)



(b)

Non-Linear RNN(EGG, minGRU 변형 모델)의 Integer-Only Pretraining 진행 시, Population Size는 262144. 한번의 업데이트에서 262144개의 Perturbation을 처리.

모델은 PPO의 3배 속도로 학습.

기존의 ES 방법론 중 하나였던 OpenES에 비해 200배 이상 빠른 학습 속도.

Experiments

Reinforcement Learning Tasks

일반적인 Policy 모델의 훈련을 기준으로,
PPO 성능을 1.0으로 했을 시 그래프.

PPO에 비해 빠른 트레이닝 속도를 가지나,
PPO와 비등한/ 더 우월한 결과를 낼 수 있다.

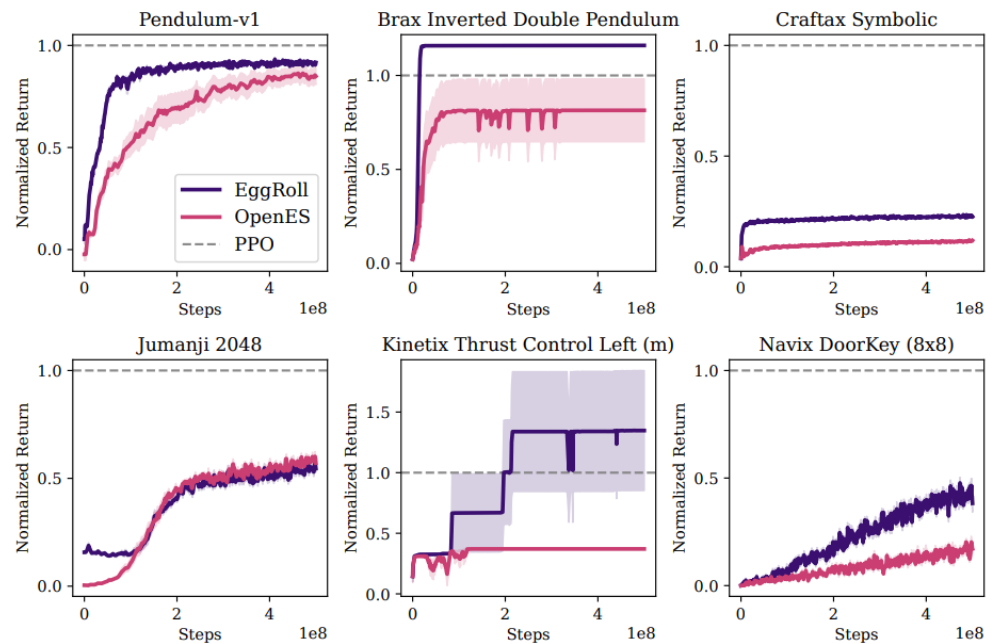
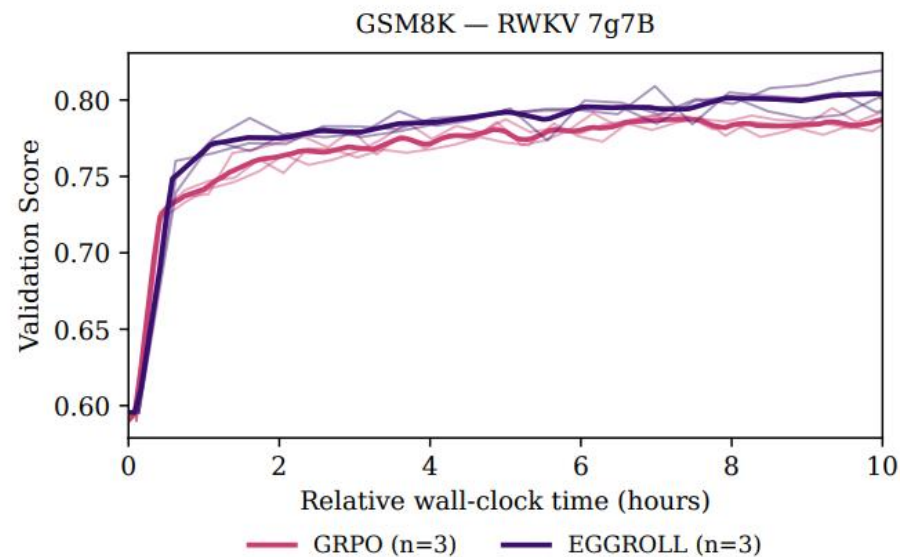
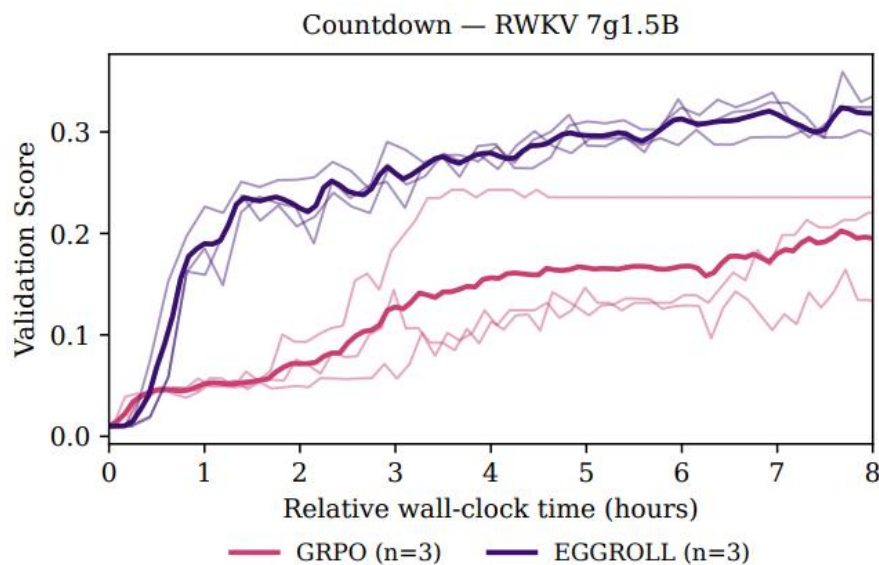


Figure 4: Comparison of reinforcement learning Mean returns normalized by PPO performance for 10 seeds. The returns are evaluated using the mean of the parameters. HPO was conducted for each algorithm/environment pair. The shaded region is the standard error of the mean .

Experiments

Finetuning LLM for Reasoning Tasks



RWKV 1.5B를 하나의 GPU에서, EGGROLL과 GRPO로 학습했다.

GRPO (32개 병렬) vs EGGROLL (1024개 병렬)

RWKV 7B는 8개 GPU에서, GRPO (256) vs EGGROLL (8096).

GRPO보다 더 빠른 학습 속도 및 효율을 보인다.

Conclusion / Takeaway

GD와 BP에 대한 대안

모델은 커져가는데, GPU는 한정적.

GD 기반 학습에는 자원적/이론적 한계가 존재한다.

기존의 ES는 더 큰 자원과 느린 속도로 쓸 수 없었지만,

EGGROLL은 더 작은 자원과 더 빠른 속도를 자랑한다.

전 구간 미분 가능성 요구에 대한 기존의 Neural Network 설계의 한계점 또한 완화 가능.

ES로 LLM을 훈련시킨 것은 처음이며, 주의 깊게 봐야 할 성과.

수렴 속도에 대한 수학적 증명과, 이론적 기반이 단단한 논문.

감사합니다